

Documentação do Projeto InterFatec

Arquivo: Controllers\AgendamentosController.cs

Classe AgendamentosController: Controlador responsável por receber requisições HTTP, processar entradas e retornar as views correspondentes.

Métodos / Funções:

Método Index:

- O que faz: Recupera e lista os dados registrados no sistema ou exibe a tela principal.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'Task<IActionResult>'.

Método Detalhes:

- O que faz: Executa a operação ou lógica associada a 'Detalhes'.
- Recebe: int id.
- Retorna: O tipo 'Task<IActionResult>'.

Método Criar:

- O que faz: Executa a operação ou lógica associada a 'Criar'.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'Task<IActionResult>'.

Método Criar:

- O que faz: Executa a operação ou lógica associada a 'Criar'.
- Recebe: AgendamentoRegistroViewModel viewModel.
- Retorna: O tipo 'Task<IActionResult>'.

Método Editar:

- O que faz: Atualiza os dados de um registro já existente.
- Recebe: int id.
- Retorna: O tipo 'Task<IActionResult>'.

Método Editar:

- O que faz: Atualiza os dados de um registro já existente.
- Recebe: int id, AgendamentoEditarViewModel viewModel.
- Retorna: O tipo 'Task<IActionResult>'.

Método Excluir:

- O que faz: Executa a operação ou lógica associada a 'Excluir'.
- Recebe: int id.
- Retorna: O tipo 'Task<IActionResult>'.

Método ConfirmarExclusao:

- O que faz: Executa a operação ou lógica associada a 'ConfirmarExclusao'.
- Recebe: int id.
- Retorna: O tipo 'Task<IActionResult>'.

Método Cancelar:

- O que faz: Executa a operação ou lógica associada a 'Cancelar'.
- Recebe: int id.
- Retorna: O tipo 'Task<IActionResult>'.

Método PrepararListasAsync:

- O que faz: Executa a operação ou lógica associada a 'PrepararListasAsync'.
- Recebe: AgendamentoRegistroViewModel viewModel.

- Retorna: O tipo 'Task'.

Método Agendar:

- O que faz: Executa a operação ou lógica associada a 'Agendar'.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'Task<IActionResult>'.

Método Agendar:

- O que faz: Executa a operação ou lógica associada a 'Agendar'.
- Recebe: AgendamentoClienteViewModel viewModel.
- Retorna: O tipo 'Task<IActionResult>'.

Método AgendamentoConfirmado:

- O que faz: Executa a operação ou lógica associada a 'AgendamentoConfirmado'.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'IActionResult'.

Método ObterHorariosDisponiveis:

- O que faz: Executa a operação ou lógica associada a 'ObterHorariosDisponiveis'.
- Recebe: int funcionarioId, string data.
- Retorna: O tipo 'Task<IActionResult>'.

Método PrepararListasClienteAsync:

- O que faz: Executa a operação ou lógica associada a 'PrepararListasClienteAsync'.
- Recebe: AgendamentoClienteViewModel viewModel.
- Retorna: O tipo 'Task'.

Arquivo: Controllers\CientesController.cs

Classe CientesController: Controlador responsável por receber requisições HTTP, processar entradas e retornar as views correspondentes.

Métodos / Funções:**Método Index:**

- O que faz: Recupera e lista os dados registrados no sistema ou exibe a tela principal.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'Task<IActionResult>'.

Método Detalhes:

- O que faz: Executa a operação ou lógica associada a 'Detalhes'.
- Recebe: int id.
- Retorna: O tipo 'Task<IActionResult>'.

Método Criar:

- O que faz: Executa a operação ou lógica associada a 'Criar'.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'IActionResult'.

Método Criar:

- O que faz: Executa a operação ou lógica associada a 'Criar'.
- Recebe: ClienteRegistroViewModel clienteViewModel.
- Retorna: O tipo 'Task<IActionResult>'.

Método Editar:

- O que faz: Atualiza os dados de um registro já existente.
- Recebe: int id.

- Retorna: O tipo 'Task<IActionResult>'.

Método Editar:

- O que faz: Atualiza os dados de um registro já existente.
- Recebe: int id, ClienteEditarViewModel clienteViewModel.
- Retorna: O tipo 'Task<IActionResult>'.

Método Excluir:

- O que faz: Executa a operação ou lógica associada a 'Excluir'.
- Recebe: int id.
- Retorna: O tipo 'Task<IActionResult>'.

Método ExcluirConfirmado:

- O que faz: Executa a operação ou lógica associada a 'ExcluirConfirmado'.
- Recebe: int id.
- Retorna: O tipo 'Task<IActionResult>'.

Arquivo: Controllers\EspecialidadesController.cs

Classe EspecialidadesController: Controlador responsável por receber requisições HTTP, processar entradas e retornar as views correspondentes.

Métodos / Funções:**Método Index:**

- O que faz: Recupera e lista os dados registrados no sistema ou exibe a tela principal.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'Task<IActionResult>'.

Método Criar:

- O que faz: Executa a operação ou lógica associada a 'Criar'.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'IActionResult'.

Método Criar:

- O que faz: Executa a operação ou lógica associada a 'Criar'.
- Recebe: EspecialidadeViewModel especialidadeViewModel.
- Retorna: O tipo 'Task<IActionResult>'.

Arquivo: Controllers\FuncionariosController.cs

Classe FuncionariosController: Controlador responsável por receber requisições HTTP, processar entradas e retornar as views correspondentes.

Métodos / Funções:**Método Index:**

- O que faz: Recupera e lista os dados registrados no sistema ou exibe a tela principal.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'Task<IActionResult>'.

Método Detalhes:

- O que faz: Executa a operação ou lógica associada a 'Detalhes'.
- Recebe: int id.
- Retorna: O tipo 'Task<IActionResult>'.

Método Criar:

- O que faz: Executa a operação ou lógica associada a 'Criar'.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'ActionResult'.

Método Criar:

- O que faz: Executa a operação ou lógica associada a 'Criar'.
- Recebe: FuncionarioRegistroViewModel funcionarioViewModel.
- Retorna: O tipo 'Task<ActionResult>'.

Método Editar:

- O que faz: Atualiza os dados de um registro já existente.
- Recebe: int id.
- Retorna: O tipo 'Task<ActionResult>'.

Método Editar:

- O que faz: Atualiza os dados de um registro já existente.
- Recebe: int id, FuncionarioEditarViewModel funcionarioViewModel.
- Retorna: O tipo 'Task<ActionResult>'.

Método Excluir:

- O que faz: Executa a operação ou lógica associada a 'Excluir'.
- Recebe: int id.
- Retorna: O tipo 'Task<ActionResult>'.

Método ExcluirConfirmado:

- O que faz: Executa a operação ou lógica associada a 'ExcluirConfirmado'.
- Recebe: int id.
- Retorna: O tipo 'Task<ActionResult>'.

Método PrepararEspecialidadesAsync:

- O que faz: Executa a operação ou lógica associada a 'PrepararEspecialidadesAsync'.
- Recebe: FuncionarioRegistroViewModel viewModel.
- Retorna: O tipo 'Task'.

Método PrepararEspecialidadesAsync:

- O que faz: Executa a operação ou lógica associada a 'PrepararEspecialidadesAsync'.
- Recebe: FuncionarioEditarViewModel viewModel.
- Retorna: O tipo 'Task'.

Arquivo: Controllers\HomeController.cs

Classe HomeController: Controlador responsável por receber requisições HTTP, processar entradas e retornar as views correspondentes.

Métodos / Funções:**Método Index:**

- O que faz: Recupera e lista os dados registrados no sistema ou exibe a tela principal.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'ActionResult'.

Arquivo: Controllers\MarcasController.cs

Classe MarcasController: Controlador responsável por receber requisições HTTP, processar entradas e retornar as views correspondentes.

Métodos / Funções:

Método Index:

- O que faz: Recupera e lista os dados registrados no sistema ou exibe a tela principal.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'ActionResult'.

Método Criar:

- O que faz: Executa a operação ou lógica associada a 'Criar'.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'ActionResult'.

Método Criar:

- O que faz: Executa a operação ou lógica associada a 'Criar'.
- Recebe: MarcasViewModel marcaViewModel.
- Retorna: O tipo 'Task<ActionResult>'.

Método Editar:

- O que faz: Atualiza os dados de um registro já existente.
- Recebe: int id.
- Retorna: O tipo 'Task<ActionResult>'.

Método Editar:

- O que faz: Atualiza os dados de um registro já existente.
- Recebe: MarcasViewModel marcaViewModel.
- Retorna: O tipo 'Task<ActionResult>'.

Método Excluir:

- O que faz: Executa a operação ou lógica associada a 'Excluir'.
- Recebe: int id.
- Retorna: O tipo 'Task<ActionResult>'.

Método ConfirmarExclusao:

- O que faz: Executa a operação ou lógica associada a 'ConfirmarExclusao'.
- Recebe: int id.
- Retorna: O tipo 'Task<ActionResult>'.

Arquivo: Controllers\ProdutosController.cs

Classe ProdutosController: Controlador responsável por receber requisições HTTP, processar entradas e retornar as views correspondentes.

Métodos / Funções:

Método Index:

- O que faz: Recupera e lista os dados registrados no sistema ou exibe a tela principal.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'Task<ActionResult>'.

Método Detalhes:

- O que faz: Executa a operação ou lógica associada a 'Detalhes'.
- Recebe: int id.
- Retorna: O tipo 'Task<ActionResult>'.

Método Criar:

- O que faz: Executa a operação ou lógica associada a 'Criar'.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'Task<ActionResult>'.

Método Criar:

- O que faz: Executa a operação ou lógica associada a 'Criar'.
- Recebe: ProdutoRegistroViewModel viewModel.
- Retorna: O tipo 'Task<IActionResult>'.

Método Editar:

- O que faz: Atualiza os dados de um registro já existente.
- Recebe: int id.
- Retorna: O tipo 'Task<IActionResult>'.

Método Editar:

- O que faz: Atualiza os dados de um registro já existente.
- Recebe: ProdutoEditarViewModel viewModel.
- Retorna: O tipo 'Task<IActionResult>'.

Método Excluir:

- O que faz: Executa a operação ou lógica associada a 'Excluir'.
- Recebe: int id.
- Retorna: O tipo 'Task<IActionResult>'.

Método ConfirmarExclusao:

- O que faz: Executa a operação ou lógica associada a 'ConfirmarExclusao'.
- Recebe: int id.
- Retorna: O tipo 'Task<IActionResult>'.

Arquivo: Controllers\ServicosController.cs

Classe ServicosController: Controlador responsável por receber requisições HTTP, processar entradas e retornar as views correspondentes.

Métodos / Funções:**Método Index:**

- O que faz: Recupera e lista os dados registrados no sistema ou exibe a tela principal.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'Task<IActionResult>'.

Método Detalhes:

- O que faz: Executa a operação ou lógica associada a 'Detalhes'.
- Recebe: int id.
- Retorna: O tipo 'Task<IActionResult>'.

Método Criar:

- O que faz: Executa a operação ou lógica associada a 'Criar'.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'IActionResult'.

Método Criar:

- O que faz: Executa a operação ou lógica associada a 'Criar'.
- Recebe: ServicoRegistroViewModel viewModel.
- Retorna: O tipo 'Task<IActionResult>'.

Método Editar:

- O que faz: Atualiza os dados de um registro já existente.
- Recebe: int id.
- Retorna: O tipo 'Task<IActionResult>'.

Método Editar:

- O que faz: Atualiza os dados de um registro já existente.
- Recebe: `ServicoEditarViewModel viewModel`.
- Retorna: O tipo `'Task<IActionResult>'`.

Método Excluir:

- O que faz: Executa a operação ou lógica associada a 'Excluir'.
- Recebe: `int id`.
- Retorna: O tipo `'Task<IActionResult>'`.

Método ConfirmarExclusao:

- O que faz: Executa a operação ou lógica associada a 'ConfirmarExclusao'.
- Recebe: `int id`.
- Retorna: O tipo `'Task<IActionResult>'`.

Arquivo: Data\AppDbContext.cs

Classe `AppDbContext`: Modelo de dados que representa a entidade `AppDbContext` no sistema.

Propriedades / Atributos:

- Propriedade **Pessoas** (`DbSet<Pessoas>`): Armazena ou recupera a informação de Pessoas.
- Propriedade **Clientes** (`DbSet<Clientes>`): Armazena ou recupera a informação de Clientes.
- Propriedade **Funcionarios** (`DbSet<Funcionarios>`): Armazena ou recupera a informação de Funcionarios.
- Propriedade **Marcas** (`DbSet<Marcas>`): Armazena ou recupera a informação de Marcas.
- Propriedade **Especialidades** (`DbSet<Especialidades>`): Armazena ou recupera a informação de Especialidades.
- Propriedade **Produtos** (`DbSet<Produtos>`): Armazena ou recupera a informação de Produtos.
- Propriedade **Servicos** (`DbSet<Servicos>`): Armazena ou recupera a informação de Servicos.
- Propriedade **Agendamentos** (`DbSet<Agendamentos>`): Armazena ou recupera a informação de Agendamentos.
- Propriedade **Servicos_Agendados** (`DbSet<Servicos_Agendados>`): Armazena ou recupera a informação de `Servicos_Agendados`.
- Propriedade **Produtos_Agendados** (`DbSet<Produtos_Agendados>`): Armazena ou recupera a informação de `Produtos_Agendados`.

Métodos / Funções:

Método OnModelCreating:

- O que faz: Executa a operação ou lógica associada a 'OnModelCreating'.
- Recebe: `ModelBuilder modelBuilder`.
- Retorna: O tipo `'void'`.

Arquivo: Enums\AgendamentoStatus.cs

Enumeração `AgendamentoStatus`: Define um conjunto de valores constantes e nomeados usados no sistema.

Arquivo: Enums\EspecialidadeStatus.cs

Enumeração `EspecialidadeStatus`: Define um conjunto de valores constantes e nomeados usados no sistema.

Arquivo: Enums\MarcaStatus.cs

Enumeração MarcaStatus: Define um conjunto de valores constantes e nomeados usados no sistema.

Arquivo: Enums\PessoaStatus.cs

Enumeração PessoaStatus: Define um conjunto de valores constantes e nomeados usados no sistema.

Arquivo: Enums\ProdutoStatus.cs

Enumeração ProdutoStatus: Define um conjunto de valores constantes e nomeados usados no sistema.

Arquivo: Enums\ServicoStatus.cs

Enumeração ServicoStatus: Define um conjunto de valores constantes e nomeados usados no sistema.

Arquivo: Mappings\AgendamentosProfile.cs

Classe AgendamentosProfile: Configuração do AutoMapper para mapear dados entre Models e ViewModels.

Arquivo: Mappings\ClientesProfile.cs

Classe ClientesProfile: Configuração do AutoMapper para mapear dados entre Models e ViewModels.

Arquivo: Mappings\EspecialidadesProfile.cs

Classe EspecialidadesProfile: Configuração do AutoMapper para mapear dados entre Models e ViewModels.

Arquivo: Mappings\FuncionariosProfile.cs

Classe FuncionariosProfile: Configuração do AutoMapper para mapear dados entre Models e ViewModels.

Arquivo: Mappings\MarcasProfile.cs

Classe MarcasProfile: Configuração do AutoMapper para mapear dados entre Models e ViewModels.

Arquivo: Mappings\ProdutosProfile.cs

Classe ProdutosProfile: Configuração do AutoMapper para mapear dados entre Models e ViewModels.

Arquivo: Mappings\ServicosProfile.cs

Classe ServicosProfile: Configuração do AutoMapper para mapear dados entre Models e ViewModels.

Arquivo: Models\Agendamentos.cs

Classe Agendamentos: Modelo de dados que representa a entidade Agendamentos no sistema.

Propriedades / Atributos:

- Propriedade **Nr** (int): Armazena ou recupera a informação de Nr.
- Propriedade **Data** (DateTime): Armazena ou recupera a informação de Data.
- Propriedade **Total** (decimal): Armazena ou recupera a informação de Total.
- Propriedade **Status** (AgendamentoStatus): Armazena ou recupera a informação de Status.
- Propriedade **Clienteld** (int): Chave estrangeira de relacionamento.
- Propriedade **Cliente** (Clientes?): Armazena ou recupera a informação de Cliente.
- Propriedade **Servicos_Agendados** (ICollection<Servicos_Agendados>): Armazena ou recupera a informação de Servicos_Agendados.
- Propriedade **Produtos_Agendados** (ICollection<Produtos_Agendados>): Armazena ou recupera a informação de Produtos_Agendados.

Arquivo: Models\Clientes.cs

Classe Clientes: Modelo de dados que representa a entidade Clientes no sistema.

Propriedades / Atributos:

- Propriedade **Agendamentos** (ICollection<Agendamentos>): Armazena ou recupera a informação de Agendamentos.

Arquivo: Models\Especialidades.cs

Classe Especialidades: Modelo de dados que representa a entidade Especialidades no sistema.

Propriedades / Atributos:

- Propriedade **Id** (int): Identificador único do registro.
- Propriedade **Descricao** (string): Armazena ou recupera a informação de Descricao.
- Propriedade **Status** (EspecialidadeStatus): Armazena ou recupera a informação de Status.
- Propriedade **Funcionarios** (ICollection<Funcionarios>): Armazena ou recupera a informação de Funcionarios.

Arquivo: Models\Funcionarios.cs

Classe Funcionarios: Modelo de dados que representa a entidade Funcionarios no sistema.

Propriedades / Atributos:

- Propriedade **Salario** (decimal): Armazena ou recupera a informação de Salario.
- Propriedade **Especialidadeld** (int): Chave estrangeira de relacionamento.
- Propriedade **Especialidade** (Especialidades?): Armazena ou recupera a informação de Especialidade.
- Propriedade **Servicos_Agendados** (ICollection<Servicos_Agendados>): Armazena ou recupera a informação de Servicos_Agendados.

Arquivo: Models\Marcas.cs

Classe Marcas: Modelo de dados que representa a entidade Marcas no sistema.

Propriedades / Atributos:

- Propriedade **Id** (int): Identificador único do registro.

- Propriedade **Nome** (string): Armazena ou recupera a informação de Nome.
- Propriedade **Status** (MarcaStatus): Armazena ou recupera a informação de Status.
- Propriedade **Produtos** (ICollection<Produtos>): Armazena ou recupera a informação de Produtos.

Arquivo: Models\Pessoas.cs

Classe Pessoas: Modelo de dados que representa a entidade Pessoas no sistema.

Propriedades / Atributos:

- Propriedade **Id** (int): Identificador único do registro.
- Propriedade **Nome** (string): Armazena ou recupera a informação de Nome.
- Propriedade **Telefone** (string): Armazena ou recupera a informação de Telefone.
- Propriedade **Status** (PessoaStatus): Armazena ou recupera a informação de Status.

Arquivo: Models\Produtos.cs

Classe Produtos: Modelo de dados que representa a entidade Produtos no sistema.

Propriedades / Atributos:

- Propriedade **Codigo** (int): Armazena ou recupera a informação de Codigo.
- Propriedade **Nome** (string): Armazena ou recupera a informação de Nome.
- Propriedade **Status** (ProdutoStatus): Armazena ou recupera a informação de Status.
- Propriedade **MarcId** (int): Chave estrangeira de relacionamento.
- Propriedade **Marca** (Marcas?): Armazena ou recupera a informação de Marca.
- Propriedade **Preco** (decimal): Armazena ou recupera a informação de Preco.
- Propriedade **PathImagem** (string): Armazena ou recupera a informação de PathImagem.
- Propriedade **Produtos_Agendados** (ICollection<Produtos_Agendados>): Armazena ou recupera a informação de Produtos_Agendados.

Arquivo: Models\Produtos_Agendados.cs

Classe Produtos_Agendados: Modelo de dados que representa a entidade Produtos_Agendados no sistema.

Propriedades / Atributos:

- Propriedade **Preco** (decimal): Armazena ou recupera a informação de Preco.
- Propriedade **AgendamentoNr** (int): Armazena ou recupera a informação de AgendamentoNr.
- Propriedade **Agendamento** (Agendamentos?): Armazena ou recupera a informação de Agendamento.
- Propriedade **ServicId** (int): Chave estrangeira de relacionamento.
- Propriedade **ServicoAgendado** (Servicos_Agendados?): Armazena ou recupera a informação de ServicoAgendado.
- Propriedade **ProdutoCodigo** (int): Armazena ou recupera a informação de ProdutoCodigo.
- Propriedade **Produto** (Produtos?): Armazena ou recupera a informação de Produto.

Arquivo: Models\Servicos.cs

Classe Servicos: Modelo de dados que representa a entidade Servicos no sistema.

Propriedades / Atributos:

- Propriedade **Id** (int): Identificador único do registro.
- Propriedade **Preco** (decimal): Armazena ou recupera a informação de Preco.
- Propriedade **Descricao** (string): Armazena ou recupera a informação de Descricao.
- Propriedade **Tempo** (TimeSpan): Armazena ou recupera a informação de Tempo.
- Propriedade **Status** (ServicoStatus): Armazena ou recupera a informação de Status.
- Propriedade **Servicos_Agendados** (ICollection<Servicos_Agendados>): Armazena ou recupera a informação de Servicos_Agendados.

Arquivo: Models\Servicos_Agendados.cs

Classe Servicos_Agendados: Modelo de dados que representa a entidade Servicos_Agendados no sistema.

Propriedades / Atributos:

- Propriedade **Obs** (string): Armazena ou recupera a informação de Obs.
- Propriedade **Horario** (TimeSpan): Armazena ou recupera a informação de Horario.
- Propriedade **Valor** (decimal): Armazena ou recupera a informação de Valor.
- Propriedade **AgendamentoNr** (int): Armazena ou recupera a informação de AgendamentoNr.
- Propriedade **Agendamento** (Agendamentos?): Armazena ou recupera a informação de Agendamento.
- Propriedade **Servicold** (int): Chave estrangeira de relacionamento.
- Propriedade **Servico** (Servicos?): Armazena ou recupera a informação de Servico.
- Propriedade **FuncionarioId** (int): Chave estrangeira de relacionamento.
- Propriedade **Funcionario** (Funcionarios?): Armazena ou recupera a informação de Funcionario.
- Propriedade **Produtos_Agendados** (ICollection<Produtos_Agendados>): Armazena ou recupera a informação de Produtos_Agendados.

Arquivo: Models\Sq\Views\FuncionarioProducaoView.cs

Classe FuncionarioProducaoView: Modelo de dados que representa a entidade FuncionarioProducaoView no sistema.

Propriedades / Atributos:

- Propriedade **Funcionario** (string): Armazena ou recupera a informação de Funcionario.
- Propriedade **TotalServicos** (int): Armazena ou recupera a informação de TotalServicos.

Arquivo: Models\Sq\Views\ListaAgendamentosView.cs

Classe ListaAgendamentosView: Modelo de dados que representa a entidade ListaAgendamentosView no sistema.

Propriedades / Atributos:

- Propriedade **NumeroAgendamento** (int): Armazena ou recupera a informação de NumeroAgendamento.
- Propriedade **Data** (DateTime): Armazena ou recupera a informação de Data.
- Propriedade **Total** (decimal): Armazena ou recupera a informação de Total.
- Propriedade **Cliente** (string): Armazena ou recupera a informação de Cliente.
- Propriedade **Status** (int): Armazena ou recupera a informação de Status.
- Propriedade **StatusDescricao** (string): Armazena ou recupera a informação de StatusDescricao.

Arquivo: Models\Sq\Views\ListaClientesView.cs

Classe ListaClientesView: Modelo de dados que representa a entidade ListaClientesView no sistema.

Propriedades / Atributos:

- Propriedade **Id** (int): Identificador único do registro.
- Propriedade **Nome** (string): Armazena ou recupera a informação de Nome.
- Propriedade **Telefone** (string): Armazena ou recupera a informação de Telefone.
- Propriedade **StatusId** (int): Chave estrangeira de relacionamento.
- Propriedade **StatusDescricao** (string): Armazena ou recupera a informação de StatusDescricao.

Arquivo: Models\SqlViews\ListaEspecialidadesView.cs

Classe ListaEspecialidadesView: Modelo de dados que representa a entidade ListaEspecialidadesView no sistema.

Propriedades / Atributos:

- Propriedade **Especialidade** (string): Armazena ou recupera a informação de Especialidade.
- Propriedade **TotalFuncionarios** (int): Armazena ou recupera a informação de TotalFuncionarios.

Arquivo: Models\SqlViews\ListaFuncionariosView.cs

Classe ListaFuncionariosView: Modelo de dados que representa a entidade ListaFuncionariosView no sistema.

Propriedades / Atributos:

- Propriedade **Id** (int): Identificador único do registro.
- Propriedade **Nome** (string): Armazena ou recupera a informação de Nome.
- Propriedade **Telefone** (string): Armazena ou recupera a informação de Telefone.
- Propriedade **Especialidade** (string): Armazena ou recupera a informação de Especialidade.
- Propriedade **Salario** (decimal): Armazena ou recupera a informação de Salario.
- Propriedade **StatusId** (int): Chave estrangeira de relacionamento.
- Propriedade **StatusDescricao** (string): Armazena ou recupera a informação de StatusDescricao.

Arquivo: Models\SqlViews\ListaMarcasView.cs

Classe ListaMarcasView: Modelo de dados que representa a entidade ListaMarcasView no sistema.

Propriedades / Atributos:

- Propriedade **Id** (int): Identificador único do registro.
- Propriedade **Nome** (string): Armazena ou recupera a informação de Nome.
- Propriedade **Status** (int): Armazena ou recupera a informação de Status.
- Propriedade **StatusDescricao** (string): Armazena ou recupera a informação de StatusDescricao.

Arquivo: Models\SqlViews\ListaProdutoAgendamentoView.cs

Classe ListaProdutoAgendamentoView: Modelo de dados que representa a entidade ListaProdutoAgendamentoView no sistema.

Propriedades / Atributos:

- Propriedade **NumeroAgendamento** (int): Armazena ou recupera a informação de NumeroAgendamento.

- Propriedade **NomeServico** (string): Armazena ou recupera a informação de NomeServico.
- Propriedade **Observacao** (string): Armazena ou recupera a informação de Observacao.
- Propriedade **NomeProduto** (string): Armazena ou recupera a informação de NomeProduto.
- Propriedade **Marca** (string): Armazena ou recupera a informação de Marca.
- Propriedade **Preco** (decimal): Armazena ou recupera a informação de Preco.

Arquivo: Models\SqlViews\ListaProdutosView.cs

Classe ListaProdutosView: Modelo de dados que representa a entidade ListaProdutosView no sistema.

Propriedades / Atributos:

- Propriedade **Codigo** (int): Armazena ou recupera a informação de Codigo.
- Propriedade **Nome** (string): Armazena ou recupera a informação de Nome.
- Propriedade **Marca** (string): Armazena ou recupera a informação de Marca.
- Propriedade **Preco** (decimal): Armazena ou recupera a informação de Preco.
- Propriedade **PathImagem** (string): Armazena ou recupera a informação de PathImagem.
- Propriedade **StatusId** (int): Chave estrangeira de relacionamento.
- Propriedade **StatusDescricao** (string): Armazena ou recupera a informação de StatusDescricao.

Arquivo: Models\SqlViews\ListaServicoAgendamentoView.cs

Classe ListaServicoAgendamentoView: Modelo de dados que representa a entidade ListaServicoAgendamentoView no sistema.

Propriedades / Atributos:

- Propriedade **NumeroAgendamento** (int): Armazena ou recupera a informação de NumeroAgendamento.
- Propriedade **NomeServico** (string): Armazena ou recupera a informação de NomeServico.
- Propriedade **Observacao** (string): Armazena ou recupera a informação de Observacao.
- Propriedade **Horario** (TimeSpan): Armazena ou recupera a informação de Horario.
- Propriedade **Funcionario** (string): Armazena ou recupera a informação de Funcionario.
- Propriedade **Valor** (decimal): Armazena ou recupera a informação de Valor.

Arquivo: Models\SqlViews\ListaServicosView.cs

Classe ListaServicosView: Modelo de dados que representa a entidade ListaServicosView no sistema.

Propriedades / Atributos:

- Propriedade **Id** (int): Identificador único do registro.
- Propriedade **Descricao** (string): Armazena ou recupera a informação de Descricao.
- Propriedade **Preco** (decimal): Armazena ou recupera a informação de Preco.
- Propriedade **Tempo** (TimeSpan): Armazena ou recupera a informação de Tempo.
- Propriedade **StatusId** (int): Chave estrangeira de relacionamento.
- Propriedade **StatusDescricao** (string): Armazena ou recupera a informação de StatusDescricao.

Arquivo: Models\SqlViews\ProdutosPorMarcaView.cs

Classe ProdutosPorMarcaView: Modelo de dados que representa a entidade ProdutosPorMarcaView no sistema.

Propriedades / Atributos:

- Propriedade **Marca** (string): Armazena ou recupera a informação de Marca.
- Propriedade **TotalProdutos** (int): Armazena ou recupera a informação de TotalProdutos.

Arquivo: Repositories\AgendamentosRepository.cs

Classe AgendamentosRepository: Repositório responsável pela comunicação com o banco de dados (operações CRUD).

Métodos / Funções:

Método ObterTodosAgendamentos:

- O que faz: Executa a operação ou lógica associada a 'ObterTodosAgendamentos'.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'Task<IEnumerable<Agendamentos>>'.

Método CriarAgendamento:

- O que faz: Executa a operação ou lógica associada a 'CriarAgendamento'.
- Recebe: Agendamentos agendamento.
- Retorna: O tipo 'Task'.

Método AtualizarAgendamento:

- O que faz: Executa a operação ou lógica associada a 'AtualizarAgendamento'.
- Recebe: Agendamentos agendamento.
- Retorna: O tipo 'Task'.

Método ExcluirAgendamento:

- O que faz: Executa a operação ou lógica associada a 'ExcluirAgendamento'.
- Recebe: Agendamentos agendamento.
- Retorna: O tipo 'Task'.

Método ObterHorariosOcupados:

- O que faz: Executa a operação ou lógica associada a 'ObterHorariosOcupados'.
- Recebe: int funcionarioId, DateTime data.
- Retorna: O tipo 'Task<List<TimeSpan>>'.

Arquivo: Repositories\ClientesRepository.cs

Classe ClientesRepository: Repositório responsável pela comunicação com o banco de dados (operações CRUD).

Métodos / Funções:

Método ObterTodosClientes:

- O que faz: Executa a operação ou lógica associada a 'ObterTodosClientes'.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'Task<IEnumerable<Clientes>>'.

Método RegistrarCliente:

- O que faz: Executa a operação ou lógica associada a 'RegistrarCliente'.
- Recebe: Clientes cliente.
- Retorna: O tipo 'Task'.

Método AtualizarCliente:

- O que faz: Executa a operação ou lógica associada a 'AtualizarCliente'.
- Recebe: Clientes cliente.
- Retorna: O tipo 'Task'.

Método ExcluirCliente:

- O que faz: Executa a operação ou lógica associada a 'ExcluirCliente'.
- Recebe: Clientes cliente.
- Retorna: O tipo 'Task'.

Método CriarCliente:

- O que faz: Executa a operação ou lógica associada a 'CriarCliente'.
- Recebe: ListaClientesView cliente, IReadOnlyDictionary<int, List<Agendamentos>>> agendamentosPorCliente.
- Retorna: O tipo 'Clientes'.

Arquivo: Repositories\EspecialidadesRepository.cs

Classe EspecialidadesRepository: Repositório responsável pela comunicação com o banco de dados (operações CRUD).

Métodos / Funções:**Método ObterTodasEspecialidades:**

- O que faz: Executa a operação ou lógica associada a 'ObterTodasEspecialidades'.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'Task<IEnumerable<Especialidades>>'.

Método CriarEspecialidade:

- O que faz: Executa a operação ou lógica associada a 'CriarEspecialidade'.
- Recebe: Especialidades especialidade.
- Retorna: O tipo 'Task'.

Método AtualizarEspecialidade:

- O que faz: Executa a operação ou lógica associada a 'AtualizarEspecialidade'.
- Recebe: Especialidades especialidade.
- Retorna: O tipo 'Task'.

Método ExcluirEspecialidade:

- O que faz: Executa a operação ou lógica associada a 'ExcluirEspecialidade'.
- Recebe: Especialidades especialidade.
- Retorna: O tipo 'Task'.

Arquivo: Repositories\FuncionariosRepository.cs

Classe FuncionariosRepository: Repositório responsável pela comunicação com o banco de dados (operações CRUD).

Métodos / Funções:**Método ObterTodosFuncionarios:**

- O que faz: Executa a operação ou lógica associada a 'ObterTodosFuncionarios'.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'Task<IEnumerable<Funcionarios>>'.

Método RegistrarFuncionario:

- O que faz: Executa a operação ou lógica associada a 'RegistrarFuncionario'.
- Recebe: Funcionarios funcionario.
- Retorna: O tipo 'Task'.

Método AtualizarFuncionario:

- O que faz: Executa a operação ou lógica associada a 'AtualizarFuncionario'.
- Recebe: Funcionarios funcionario.
- Retorna: O tipo 'Task'.

Método ExcluirFuncionario:

- O que faz: Executa a operação ou lógica associada a 'ExcluirFuncionario'.
- Recebe: Funcionarios funcionario.
- Retorna: O tipo 'Task'.

Arquivo: Repositories\Interfaces\IAgendamentosRepository.cs

Interface IAgendamentosRepository: Define um contrato que as classes que a implementam devem seguir.

Arquivo: Repositories\Interfaces\IClientesRepository.cs

Interface IClientesRepository: Define um contrato que as classes que a implementam devem seguir.

Arquivo: Repositories\Interfaces\IEspecialidadesRepository.cs

Interface IEspecialidadeRepository: Define um contrato que as classes que a implementam devem seguir.

Arquivo: Repositories\Interfaces\IFuncionariosRepository.cs

Interface IFuncionariosRepository: Define um contrato que as classes que a implementam devem seguir.

Arquivo: Repositories\Interfaces\IMarcasRepository.cs

Interface IMarcasRepository: Define um contrato que as classes que a implementam devem seguir.

Arquivo: Repositories\Interfaces\IProdutosRepository.cs

Interface IProdutosRepository: Define um contrato que as classes que a implementam devem seguir.

Arquivo: Repositories\Interfaces\IServicosRepository.cs

Interface IServicosRepository: Define um contrato que as classes que a implementam devem seguir.

Arquivo: Repositories\MarcasRepository.cs

Classe MarcasRepository: Repositório responsável pela comunicação com o banco de dados (operações CRUD).

Métodos / Funções:**Método ObterTodasMarcas:**

- O que faz: Executa a operação ou lógica associada a 'ObterTodasMarcas'.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'Task<IEnumerable<Marcas>>'.

Método CriarMarca:

- O que faz: Executa a operação ou lógica associada a 'CriarMarca'.
- Recebe: Marcas marca.
- Retorna: O tipo 'Task'.

Método AtualizarMarca:

- O que faz: Executa a operação ou lógica associada a 'AtualizarMarca'.
- Recebe: Marcas marca.
- Retorna: O tipo 'Task'.

Método ExcluirMarca:

- O que faz: Executa a operação ou lógica associada a 'ExcluirMarca'.
- Recebe: Marcas marca.
- Retorna: O tipo 'Task'.

Arquivo: Repositories\ProdutosRepository.cs

Classe ProdutoRepository: Repositório responsável pela comunicação com o banco de dados (operações CRUD).

Métodos / Funções:**Método ObterTodosProdutos:**

- O que faz: Executa a operação ou lógica associada a 'ObterTodosProdutos'.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'Task<IEnumerable<Produtos>>'.

Método CriarProduto:

- O que faz: Executa a operação ou lógica associada a 'CriarProduto'.
- Recebe: Produtos produto.
- Retorna: O tipo 'Task'.

Método AtualizarProduto:

- O que faz: Executa a operação ou lógica associada a 'AtualizarProduto'.
- Recebe: Produtos produto.
- Retorna: O tipo 'Task'.

Método ExcluirProduto:

- O que faz: Executa a operação ou lógica associada a 'ExcluirProduto'.
- Recebe: Produtos produto.
- Retorna: O tipo 'Task'.

Arquivo: Repositories\ServicosRepository.cs

Classe ServicosRepository: Repositório responsável pela comunicação com o banco de dados (operações CRUD).

Métodos / Funções:**Método ObterTodosServicos:**

- O que faz: Executa a operação ou lógica associada a 'ObterTodosServicos'.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'Task<IEnumerable<Servicos>>'.

Método CriarServico:

- O que faz: Executa a operação ou lógica associada a 'CriarServico'.

- Recebe: Servicos servico.
- Retorna: O tipo 'Task'.

Método AtualizarServico:

- O que faz: Executa a operação ou lógica associada a 'AtualizarServico'.
- Recebe: Servicos servico.
- Retorna: O tipo 'Task'.

Método ExcluirServico:

- O que faz: Executa a operação ou lógica associada a 'ExcluirServico'.
- Recebe: Servicos servico.
- Retorna: O tipo 'Task'.

Arquivo: Services\AgendamentosService.cs

Classe AgendamentosService: Serviço contendo as regras de negócio e lógica de aplicação.

Métodos / Funções:

Método ObterTodosAgendamentos:

- O que faz: Executa a operação ou lógica associada a 'ObterTodosAgendamentos'.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'Task<IEnumerable<Agendamentos>>'.

Método CriarAgendamento:

- O que faz: Executa a operação ou lógica associada a 'CriarAgendamento'.
- Recebe: AgendamentoRegistroViewModel viewModel.
- Retorna: O tipo 'Task'.

Método AtualizarAgendamento:

- O que faz: Executa a operação ou lógica associada a 'AtualizarAgendamento'.
- Recebe: AgendamentoEditarViewModel viewModel.
- Retorna: O tipo 'Task'.

Método CancelarAgendamento:

- O que faz: Executa a operação ou lógica associada a 'CancelarAgendamento'.
- Recebe: int nr.
- Retorna: O tipo 'Task'.

Método ExcluirAgendamento:

- O que faz: Executa a operação ou lógica associada a 'ExcluirAgendamento'.
- Recebe: int nr.
- Retorna: O tipo 'Task'.

Método AdicionarServicosAsync:

- O que faz: Executa a operação ou lógica associada a 'AdicionarServicosAsync'.
- Recebe: Agendamentos agendamento, IEnumerable<ServicoAgendadoViewModel>? servicosSelecioneados.
- Retorna: O tipo 'Task'.

Método AdicionarProdutosAsync:

- O que faz: Executa a operação ou lógica associada a 'AdicionarProdutosAsync'.
- Recebe: Agendamentos agendamento, IEnumerable<ProdutoAgendadoViewModel>? produtosSelecioneados.
- Retorna: O tipo 'Task'.

Método ObterClienteObrigatorioAsync:

- O que faz: Executa a operação ou lógica associada a 'ObterClienteObrigatorioAsync'.
- Recebe: int clienteld.
- Retorna: O tipo 'Task<Clientes>'.

Método ObterFuncionarioObrigatorioAsync:

- O que faz: Executa a operação ou lógica associada a 'ObterFuncionarioObrigatorioAsync'.
- Recebe: int funcionariold.
- Retorna: O tipo 'Task<Funcionarios>'.

Método ObterServicoObrigatorioAsync:

- O que faz: Executa a operação ou lógica associada a 'ObterServicoObrigatorioAsync'.
- Recebe: int servicold.
- Retorna: O tipo 'Task<Servicos>'.

Método ObterProdutoObrigatorioAsync:

- O que faz: Executa a operação ou lógica associada a 'ObterProdutoObrigatorioAsync'.
- Recebe: int produtoCodigo.
- Retorna: O tipo 'Task<Produtos>'.

Método CriarAgendamentoCliente:

- O que faz: Executa a operação ou lógica associada a 'CriarAgendamentoCliente'.
- Recebe: AgendamentoClienteViewModel viewModel.
- Retorna: O tipo 'Task'.

Método ObterHorariosDisponiveis:

- O que faz: Executa a operação ou lógica associada a 'ObterHorariosDisponiveis'.
- Recebe: int funcionariold, DateTime data.
- Retorna: O tipo 'Task<List<TimeSpan>>'.

Arquivo: Services\ClientesService.cs

Classe ClientesService: Serviço contendo as regras de negócio e lógica de aplicação.

Métodos / Funções:**Método ObterTodosClientes:**

- O que faz: Executa a operação ou lógica associada a 'ObterTodosClientes'.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'Task<IEnumerable<Clientes>>'.

Método RegistrarCliente:

- O que faz: Executa a operação ou lógica associada a 'RegistrarCliente'.
- Recebe: Clientes cliente.
- Retorna: O tipo 'Task'.

Método AtualizarCliente:

- O que faz: Executa a operação ou lógica associada a 'AtualizarCliente'.
- Recebe: Clientes cliente.
- Retorna: O tipo 'Task'.

Método ExcluirCliente:

- O que faz: Executa a operação ou lógica associada a 'ExcluirCliente'.
- Recebe: int id.
- Retorna: O tipo 'Task'.

Arquivo: Services\EspecialidadeService.cs

Classe EspecialidadeService: Serviço contendo as regras de negócio e lógica de aplicação.

Métodos / Funções:

Método ObterTodasEspecialidades:

- O que faz: Executa a operação ou lógica associada a 'ObterTodasEspecialidades'.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'Task<IEnumerable<Especialidades>>'.

Método CriarEspecialidade:

- O que faz: Executa a operação ou lógica associada a 'CriarEspecialidade'.
- Recebe: Especialidades especialidade.
- Retorna: O tipo 'Task'.

Método AtualizarEspecialidade:

- O que faz: Executa a operação ou lógica associada a 'AtualizarEspecialidade'.
- Recebe: Especialidades especialidade.
- Retorna: O tipo 'Task'.

Método ExcluirEspecialidade:

- O que faz: Executa a operação ou lógica associada a 'ExcluirEspecialidade'.
- Recebe: int id.
- Retorna: O tipo 'Task'.

Arquivo: Services\FuncionariosService.cs

Classe FuncionariosService: Serviço contendo as regras de negócio e lógica de aplicação.

Métodos / Funções:

Método ObterTodosFuncionarios:

- O que faz: Executa a operação ou lógica associada a 'ObterTodosFuncionarios'.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'Task<IEnumerable<Funcionarios>>'.

Método RegistrarFuncionario:

- O que faz: Executa a operação ou lógica associada a 'RegistrarFuncionario'.
- Recebe: Funcionarios funcionario.
- Retorna: O tipo 'Task'.

Método AtualizarFuncionario:

- O que faz: Executa a operação ou lógica associada a 'AtualizarFuncionario'.
- Recebe: Funcionarios funcionario.
- Retorna: O tipo 'Task'.

Método ExcluirFuncionario:

- O que faz: Executa a operação ou lógica associada a 'ExcluirFuncionario'.
- Recebe: int id.
- Retorna: O tipo 'Task'.

Arquivo: Services\Interfaces\IAgendamentosService.cs

Interface IAgendamentosService: Define um contrato que as classes que a implementam devem seguir.

Arquivo: Services\Interfaces\IClientesService.cs

Interface IClientesService: Define um contrato que as classes que a implementam devem seguir.

Arquivo: Services\Interfaces\IEspecialidadeService.cs

Interface IEspecialidadeService: Define um contrato que as classes que a implementam devem seguir.

Arquivo: Services\Interfaces\IFuncionariosService.cs

Interface IFuncionariosService: Define um contrato que as classes que a implementam devem seguir.

Arquivo: Services\Interfaces\IMarcasService.cs

Interface IMarcasService: Define um contrato que as classes que a implementam devem seguir.

Arquivo: Services\Interfaces\IProdutosService.cs

Interface IProdutosService: Define um contrato que as classes que a implementam devem seguir.

Arquivo: Services\Interfaces\IServicosService.cs

Interface IServicosService: Define um contrato que as classes que a implementam devem seguir.

Arquivo: Services\MarcasService.cs

Classe MarcasService: Serviço contendo as regras de negócio e lógica de aplicação.

Métodos / Funções:

Método ObterTodasMarcas:

- O que faz: Executa a operação ou lógica associada a 'ObterTodasMarcas'.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'Task<IEnumerable<Marcas>>'.

Método CriarMarca:

- O que faz: Executa a operação ou lógica associada a 'CriarMarca'.
- Recebe: Marcas marca.
- Retorna: O tipo 'Task'.

Método AtualizarMarca:

- O que faz: Executa a operação ou lógica associada a 'AtualizarMarca'.
- Recebe: Marcas marca.
- Retorna: O tipo 'Task'.

Método ExcluirMarca:

- O que faz: Executa a operação ou lógica associada a 'ExcluirMarca'.
- Recebe: int id.
- Retorna: O tipo 'Task'.

Arquivo: Services\ProdutosService.cs

Classe ProdutosService: Serviço contendo as regras de negócio e lógica de aplicação.

Métodos / Funções:

Método ObterTodosProdutos:

- O que faz: Executa a operação ou lógica associada a 'ObterTodosProdutos'.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'Task<IEnumerable<Produtos>>'.

Método CriarProduto:

- O que faz: Executa a operação ou lógica associada a 'CriarProduto'.
- Recebe: Produtos produto.
- Retorna: O tipo 'Task'.

Método AtualizarProduto:

- O que faz: Executa a operação ou lógica associada a 'AtualizarProduto'.
- Recebe: Produtos produto.
- Retorna: O tipo 'Task'.

Método ExcluirProduto:

- O que faz: Executa a operação ou lógica associada a 'ExcluirProduto'.
- Recebe: int id.
- Retorna: O tipo 'Task'.

Arquivo: Services\ServicosService.cs

Classe ServicosService: Serviço contendo as regras de negócio e lógica de aplicação.

Métodos / Funções:

Método ObterTodosServicos:

- O que faz: Executa a operação ou lógica associada a 'ObterTodosServicos'.
- Recebe: Nenhum parâmetro.
- Retorna: O tipo 'Task<IEnumerable<Servicos>>'.

Método CriarServico:

- O que faz: Executa a operação ou lógica associada a 'CriarServico'.
- Recebe: Servicos servico.
- Retorna: O tipo 'Task'.

Método AtualizarServico:

- O que faz: Executa a operação ou lógica associada a 'AtualizarServico'.
- Recebe: Servicos servico.
- Retorna: O tipo 'Task'.

Método ExcluirServico:

- O que faz: Executa a operação ou lógica associada a 'ExcluirServico'.
- Recebe: int id.
- Retorna: O tipo 'Task'.

Arquivo: ViewModels\Agendamentos\AgendamentoClienteViewModel.cs

Classe AgendamentoClienteViewModel: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **Nome** (string): Armazena ou recupera a informação de Nome.
- Propriedade **Telefone** (string): Armazena ou recupera a informação de Telefone.
- Propriedade **Data** (DateTime): Armazena ou recupera a informação de Data.
- Propriedade **Servicos** (List<ServicoClientItemViewModel>): Armazena ou recupera a informação de Servicos.
- Propriedade **ServicosList** (IEnumerable<SelectListItem>?): Armazena ou recupera a informação de ServicosList.
- Propriedade **FuncionariosList** (IEnumerable<SelectListItem>?): Armazena ou recupera a informação de FuncionariosList.
- Propriedade **ProdutosList** (IEnumerable<SelectListItem>?): Armazena ou recupera a informação de ProdutosList.

Arquivo: ViewModels\Agendamentos\AgendamentoDetalhesViewModel.cs

Classe AgendamentoDetalhesViewModel: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **Nr** (int): Armazena ou recupera a informação de Nr.
- Propriedade **ClienteNome** (string): Armazena ou recupera a informação de ClienteNome.
- Propriedade **DataHora** (DateTime): Armazena ou recupera a informação de DataHora.
- Propriedade **Status** (AgendamentoStatus): Armazena ou recupera a informação de Status.
- Propriedade **Total** (decimal): Armazena ou recupera a informação de Total.
- Propriedade **ServicosAgendados** (List<ServicoAgendadoViewModel>): Armazena ou recupera a informação de ServicosAgendados.
- Propriedade **ProdutosAgendados** (List<ProdutoAgendadoViewModel>): Armazena ou recupera a informação de ProdutosAgendados.

Arquivo: ViewModels\Agendamentos\AgendamentoEditarViewModel.cs

Classe AgendamentoEditarViewModel: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **Nr** (int): Armazena ou recupera a informação de Nr.

Arquivo: ViewModels\Agendamentos\AgendamentoListagemViewModel.cs

Classe AgendamentoListagemViewModel: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **Nr** (int): Armazena ou recupera a informação de Nr.
- Propriedade **ClienteNome** (string): Armazena ou recupera a informação de ClienteNome.
- Propriedade **DataHora** (DateTime): Armazena ou recupera a informação de DataHora.
- Propriedade **Status** (AgendamentoStatus): Armazena ou recupera a informação de Status.
- Propriedade **Total** (decimal): Armazena ou recupera a informação de Total.
- Propriedade **QuantidadeServicos** (int): Armazena ou recupera a informação de QuantidadeServicos.

- Propriedade **QuantidadeProdutos** (int): Armazena ou recupera a informação de QuantidadeProdutos.

Arquivo: ViewModels\Agendamentos\AgendamentoRegistroViewModel.cs

Classe AgendamentoRegistroViewModel: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **Clienteld** (int): Chave estrangeira de relacionamento.
- Propriedade **DataHora** (DateTime): Armazena ou recupera a informação de DataHora.
- Propriedade **Status** (AgendamentoStatus): Armazena ou recupera a informação de Status.
- Propriedade **ServicosSelecionados** (List<ServicoAgendadoViewModel>): Armazena ou recupera a informação de ServicosSelecionados.
- Propriedade **ProdutosSelecionados** (List<ProdutoAgendadoViewModel>): Armazena ou recupera a informação de ProdutosSelecionados.
- Propriedade **CientesList** (IEnumerable<SelectListItem>?): Armazena ou recupera a informação de CientesList.
- Propriedade **FuncionariosList** (IEnumerable<SelectListItem>?): Armazena ou recupera a informação de FuncionariosList.
- Propriedade **ServicosList** (IEnumerable<SelectListItem>?): Armazena ou recupera a informação de ServicosList.
- Propriedade **ProdutosList** (IEnumerable<SelectListItem>?): Armazena ou recupera a informação de ProdutosList.

Arquivo: ViewModels\Agendamentos\ProdutoAgendadoViewModel.cs

Classe ProdutoAgendadoViewModel: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **ProdutoCodigo** (int): Armazena ou recupera a informação de ProdutoCodigo.
- Propriedade **Servicoid** (int): Chave estrangeira de relacionamento.
- Propriedade **Preco** (decimal): Armazena ou recupera a informação de Preco.
- Propriedade **ProdutoNome** (string): Armazena ou recupera a informação de ProdutoNome.
- Propriedade **ServicoNome** (string): Armazena ou recupera a informação de ServicoNome.

Arquivo: ViewModels\Agendamentos\ServicoClienteltemViewModel.cs

Classe ServicoClienteltemViewModel: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **Servicoid** (int): Chave estrangeira de relacionamento.
- Propriedade **Funcionarioid** (int): Chave estrangeira de relacionamento.
- Propriedade **Horario** (TimeSpan): Armazena ou recupera a informação de Horario.
- Propriedade **Obs** (string): Armazena ou recupera a informação de Obs.
- Propriedade **ProdutosCodigos** (List<int>): Armazena ou recupera a informação de ProdutosCodigos.

Arquivo: ViewModels\Clientes\ClienteDetalhesViewModel.cs

Classe ClienteDetalhesViewModel: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **Id** (int): Identificador único do registro.
- Propriedade **TotalAgendamentos** (int): Armazena ou recupera a informação de TotalAgendamentos.
- Propriedade **DataUltimoAgendamento** (DateTime?): Armazena ou recupera a informação de DataUltimoAgendamento.

Arquivo: ViewModels\Clientes\ClienteEditarViewModel.cs

Classe ClienteEditarViewModel: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **Id** (int): Identificador único do registro.

Arquivo: ViewModels\Clientes\ClienteListagemViewModel.cs

Classe ClienteListagemViewModel: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **Id** (int): Identificador único do registro.
- Propriedade **TotalAgendamentos** (int): Armazena ou recupera a informação de TotalAgendamentos.
- Propriedade **DataUltimoAgendamento** (DateTime?): Armazena ou recupera a informação de DataUltimoAgendamento.

Arquivo: ViewModels\Clientes\ClienteRegistroViewModel.cs

Classe ClienteRegistroViewModel: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Arquivo: ViewModels\EspecialidadeViewModel.cs

Classe EspecialidadeViewModel: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **Id** (int): Identificador único do registro.
- Propriedade **Nome** (string): Armazena ou recupera a informação de Nome.
- Propriedade **Status** (EspecialidadeStatus): Armazena ou recupera a informação de Status.

Arquivo: ViewModels\Funcionarios\FuncionarioDetalhesViewModel.cs

Classe `FuncionarioDetalhesViewModel`: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **Id** (int): Identificador único do registro.
- Propriedade **Salario** (decimal): Armazena ou recupera a informação de Salario.
- Propriedade **EspecialidadeNome** (string): Armazena ou recupera a informação de EspecialidadeNome.
- Propriedade **ServicosAgendados** (ICollection<ServicoAgendadoViewModel>): Armazena ou recupera a informação de ServicosAgendados.

Arquivo: ViewModels\Funcionarios\FuncionarioEditarViewModel.cs

Classe `FuncionarioEditarViewModel`: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **Id** (int): Identificador único do registro.
- Propriedade **Salario** (decimal): Armazena ou recupera a informação de Salario.
- Propriedade **EspecialidadeId** (int): Chave estrangeira de relacionamento.
- Propriedade **EspecialidadesList** (IEnumerable<SelectedItem>): Armazena ou recupera a informação de EspecialidadesList.

Arquivo: ViewModels\Funcionarios\FuncionarioListagemViewModel.cs

Classe `FuncionarioListagemViewModel`: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **Id** (int): Identificador único do registro.
- Propriedade **Salario** (decimal): Armazena ou recupera a informação de Salario.
- Propriedade **EspecialidadeNome** (string): Armazena ou recupera a informação de EspecialidadeNome.

Arquivo: ViewModels\Funcionarios\FuncionarioRegistroViewModel.cs

Classe `FuncionarioRegistroViewModel`: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **Salario** (decimal): Armazena ou recupera a informação de Salario.
- Propriedade **EspecialidadeId** (int): Chave estrangeira de relacionamento.
- Propriedade **EspecialidadesList** (IEnumerable<SelectedItem>): Armazena ou recupera a informação de EspecialidadesList.

Arquivo: ViewModels\MarcasViewModel.cs

Classe `MarcasViewModel`: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **Id** (int): Identificador único do registro.
- Propriedade **Nome** (string): Armazena ou recupera a informação de Nome.
- Propriedade **Status** (MarcaStatus): Armazena ou recupera a informação de Status.

Arquivo: ViewModels\PessoasViewModel.cs

Classe PessoasViewModel: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **Nome** (string): Armazena ou recupera a informação de Nome.
- Propriedade **Telefone** (string): Armazena ou recupera a informação de Telefone.
- Propriedade **Status** (PessoaStatus): Armazena ou recupera a informação de Status.

Arquivo: ViewModels\Produtos\ProdutoDetalhesViewModel.cs

Classe ProdutoDetalhesViewModel: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **Codigo** (int): Armazena ou recupera a informação de Codigo.
- Propriedade **Nome** (string): Armazena ou recupera a informação de Nome.
- Propriedade **NomeMarca** (string): Armazena ou recupera a informação de NomeMarca.
- Propriedade **PrecoFormatado** (string): Armazena ou recupera a informação de PrecoFormatado.
- Propriedade **PathImagem** (string): Armazena ou recupera a informação de PathImagem.
- Propriedade **Status** (ProdutoStatus): Armazena ou recupera a informação de Status.

Arquivo: ViewModels\Produtos\ProdutoEditarViewModel.cs

Classe ProdutoEditarViewModel: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **Codigo** (int): Armazena ou recupera a informação de Codigo.
- Propriedade **Nome** (string): Armazena ou recupera a informação de Nome.
- Propriedade **Marcald** (int): Chave estrangeira de relacionamento.
- Propriedade **Preco** (decimal): Armazena ou recupera a informação de Preco.
- Propriedade **PathImagem** (string): Armazena ou recupera a informação de PathImagem.
- Propriedade **Status** (ProdutoStatus): Armazena ou recupera a informação de Status.
- Propriedade **MarcasList** (IEnumerable<SelectListItem>): Armazena ou recupera a informação de MarcasList.

Arquivo: ViewModels\Produtos\ProdutoListagemViewModel.cs

Classe ProdutoListagemViewModel: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **Codigo** (int): Armazena ou recupera a informação de Codigo.
- Propriedade **Nome** (string): Armazena ou recupera a informação de Nome.
- Propriedade **NomeMarca** (string): Armazena ou recupera a informação de NomeMarca.
- Propriedade **PrecoFormatado** (string): Armazena ou recupera a informação de PrecoFormatado.
- Propriedade **Status** (ProdutoStatus): Armazena ou recupera a informação de Status.

Arquivo: ViewModels\Produtos\ProdutoRegistroViewModel.cs

Classe ProdutoRegistroViewModel: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **Nome** (string): Armazena ou recupera a informação de Nome.
- Propriedade **MarcId** (int): Chave estrangeira de relacionamento.
- Propriedade **Preco** (decimal): Armazena ou recupera a informação de Preco.
- Propriedade **PathImagem** (string): Armazena ou recupera a informação de PathImagem.
- Propriedade **Status** (ProdutoStatus): Armazena ou recupera a informação de Status.
- Propriedade **MarcasList** (IEnumerable<SelectListItem>): Armazena ou recupera a informação de MarcasList.

Arquivo: ViewModels\ServicoAgendadoViewModel.cs

Classe ServicoAgendadoViewModel: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **ServicoId** (int): Chave estrangeira de relacionamento.
- Propriedade **FuncionarioId** (int): Chave estrangeira de relacionamento.
- Propriedade **Horario** (TimeSpan): Armazena ou recupera a informação de Horario.
- Propriedade **Obs** (string): Armazena ou recupera a informação de Obs.
- Propriedade **Valor** (decimal): Armazena ou recupera a informação de Valor.
- Propriedade **ServicoNome** (string): Armazena ou recupera a informação de ServicoNome.
- Propriedade **FuncionarioNome** (string): Armazena ou recupera a informação de FuncionarioNome.

Arquivo: ViewModels\Servicos\ServicoDetalhesViewModel.cs

Classe ServicoDetalhesViewModel: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **Id** (int): Identificador único do registro.
- Propriedade **Descricao** (string): Armazena ou recupera a informação de Descricao.
- Propriedade **PrecoFormatado** (string): Armazena ou recupera a informação de PrecoFormatado.
- Propriedade **TempoFormatado** (string): Armazena ou recupera a informação de TempoFormatado.
- Propriedade **Status** (ServicoStatus): Armazena ou recupera a informação de Status.

Arquivo: ViewModels\Servicos\ServicoEditarViewModel.cs

Classe `ServicoEditarViewModel`: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **Id** (int): Identificador único do registro.
- Propriedade **Descricao** (string): Armazena ou recupera a informação de Descricao.
- Propriedade **Preco** (decimal): Armazena ou recupera a informação de Preco.
- Propriedade **Tempo** (TimeSpan): Armazena ou recupera a informação de Tempo.
- Propriedade **Status** (ServicoStatus): Armazena ou recupera a informação de Status.

Arquivo: ViewModels\Servicos\ServicoListagemViewModel.cs

Classe `ServicoListagemViewModel`: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **Id** (int): Identificador único do registro.
- Propriedade **Descricao** (string): Armazena ou recupera a informação de Descricao.
- Propriedade **PrecoFormatado** (string): Armazena ou recupera a informação de PrecoFormatado.
- Propriedade **TempoFormatado** (string): Armazena ou recupera a informação de TempoFormatado.
- Propriedade **Status** (ServicoStatus): Armazena ou recupera a informação de Status.

Arquivo: ViewModels\Servicos\ServicoRegistroViewModel.cs

Classe `ServicoRegistroViewModel`: Modelo de visualização (ViewModel) utilizado para transferir dados formatados entre a Controller e a View.

Propriedades / Atributos:

- Propriedade **Descricao** (string): Armazena ou recupera a informação de Descricao.
- Propriedade **Preco** (decimal): Armazena ou recupera a informação de Preco.
- Propriedade **Tempo** (TimeSpan): Armazena ou recupera a informação de Tempo.
- Propriedade **Status** (ServicoStatus): Armazena ou recupera a informação de Status.